

Practical 12

Task 1: Implement the Producer–Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1  
#include <stdio.h>  
#include <pthread.h>  
#include <semaphore.h>  
#include <unistd.h>  
  
#define BUFFER_SIZE 5  
#define ITEMS 10  
  
int buffer[BUFFER_SIZE];  
int in = 0, out = 0;  
  
sem_t empty;  
sem_t full;  
pthread_mutex_t mutex;  
  
void *producer(void *arg)  
{  
    for (int i = 1; i <= ITEMS; i++) {  
        sem_wait(&empty);  
  
        pthread_mutex_lock(&mutex);  
  
        buffer[in] = i;  
        printf("Produced: %d\n", i);  
        in = (in + 1) % BUFFER_SIZE;  
  
        pthread_mutex_unlock(&mutex);  
  
        sem_post(&full);  
        usleep(100000);  
    }  
  
    return NULL;  
}  
  
void *consumer(void *arg)  
{  
    for (int i = 1; i <= ITEMS; i++) {
```

```

rohita@DESKTOP-1UHU58C: ~
GNU nano 8.7.1 p12.c *
for (int i = 1; i <= ITEMS; i++) {
    sem_wait(&full);

    pthread_mutex_lock(&mutex);

    int item = buffer[out];
    printf("Consumed: %d\n", item);
    out = (out + 1) % BUFFER_SIZE;

    pthread_mutex_unlock(&mutex);

    sem_post(&empty);
    usleep(150000);
}

return NULL;
}

int main()
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}

```

Output:

```

Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Produced: 4
Consumed: 3
Produced: 5
Consumed: 4
Produced: 6
Consumed: 5
Produced: 7
Produced: 8
Consumed: 6
Produced: 9
Consumed: 7
Produced: 10
Consumed: 8
Consumed: 9
Consumed: 10

```

Observation: Counting semaphores synchronized the producer and consumer correctly and prevented buffer overflow and underflow. Larger buffer sizes allow more items to be stored and can improve throughput by reducing waiting.

Task 2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Code:

```
rohita@DESKTOP-1UHU58C: ~  
GNU nano 8.7.1 p12t2.c *  
#include <stdio.h>  
#include <pthread.h>  
#include <unistd.h>  
  
pthread_mutex_t resource1;  
pthread_mutex_t resource2;  
  
void *thread1(void *arg)  
{  
    pthread_mutex_lock(&resource1);  
    printf("Thread 1 acquired Resource 1\n");  
  
    sleep(1);  
  
    pthread_mutex_lock(&resource2);  
    printf("Thread 1 acquired Resource 2\n");  
  
    printf("Thread 1 completed\n");  
  
    pthread_mutex_unlock(&resource2);  
    pthread_mutex_unlock(&resource1);  
  
    return NULL;  
}  
  
void *thread2(void *arg)  
{  
    pthread_mutex_lock(&resource1);  
    printf("Thread 2 acquired Resource 1\n");  
  
    pthread_mutex_lock(&resource2);  
    printf("Thread 2 acquired Resource 2\n");  
  
    printf("Thread 2 completed\n");  
  
    pthread_mutex_unlock(&resource2);  
    pthread_mutex_unlock(&resource1);  
}
```

```

        return NULL;
    }

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

```

Output:

```

Thread 1 acquired Resource 1
Thread 1 acquired Resource 2
Thread 1 completed
Thread 2 acquired Resource 1
Thread 2 acquired Resource 2
Thread 2 completed

```

Observation: Resource ordering prevents circular wait by making all threads acquire resources in the same order, avoiding deadlock and allowing successful completion.